

INTRODUCCIÓN: 8080 & 8085

Los microprocesadores Intel [8080](#) y [8085](#) fueron fundamentales en la evolución temprana de la computación, especialmente en la década de 1970. El 8080, lanzado en 1974, fue un procesador de 8 bits que rápidamente se convirtió en el estándar para computadoras personales y sistemas embebidos. Su arquitectura y conjunto de instrucciones eran relativamente simples, lo que lo hacía adecuado para aplicaciones básicas. Durante esa época, el 8080 fue utilizado en varias computadoras de la época, como la IMSAI 8080 y la Altair 8800, que son consideradas algunas de las primeras computadoras personales.

El Intel 8085, lanzado en 1976, fue una mejora sobre el 8080, pero aún mantenía la arquitectura de 8 bits. El 8085 incluía una mayor integración, con la adición de algunos circuitos internos que redujeron la necesidad de componentes externos, lo que lo hizo más económico y eficiente. Este procesador fue ampliamente utilizado en sistemas embebidos, controladores y aplicaciones de automatización industrial.

Ambos microprocesadores, 8080 y 8085, sentaron las bases para el siguiente gran avance de Intel: los procesadores 8086 y 8088. Estos modelos de 16 bits representaron un salto significativo en términos de capacidad de procesamiento y arquitectura, permitiendo mayores volúmenes de memoria y un mayor rendimiento. A pesar de las diferencias en el ancho del bus entre el 8086 y el 8088, ambos compartían el mismo conjunto de instrucciones que los anteriores procesadores de 8 bits, lo que facilitó la transición a nuevas tecnologías y garantizó la compatibilidad con programas escritos para los microprocesadores 8080 y 8085.

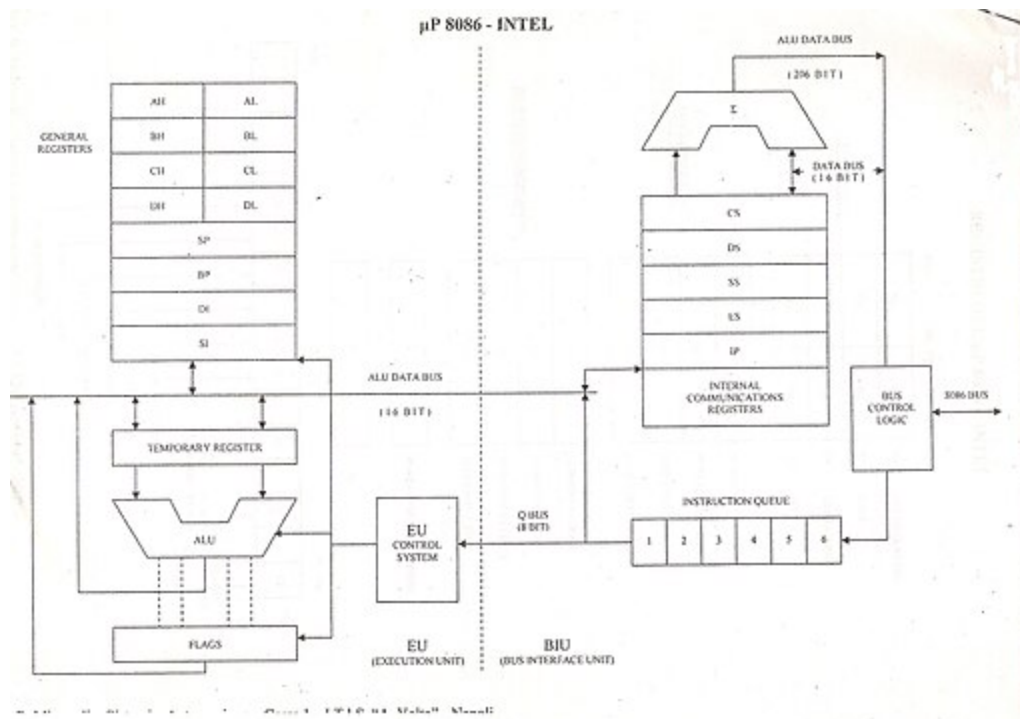
El Intel 8088, como se va a ver más adelante, triunfó sobre el 8086 en gran parte porque continuó con la compatibilidad y la arquitectura establecida por los procesadores 8080 y 8085, lo que facilitó la transición para los desarrolladores y fabricantes de hardware de la época. Mientras que el 8086, con su bus de 16 bits, ofrecía una mayor capacidad de procesamiento, el 8088 utilizaba un bus de 8 bits, lo que lo hacía más económico y más fácil de integrar con el hardware ya existente. Esta compatibilidad con las arquitecturas anteriores fue clave, ya que permitió a los fabricantes de computadoras personales aprovechar el hardware y software previamente desarrollado para los microprocesadores 8080 y 8085.

APARICIÓN DE 8086 & 8088

En 1978, Intel introdujo al mercado el microprocesador^[1] 8086 (iAPX 86), seguido en 1979 por el Intel 8088 (iAPX 88). Estos fueron los primeros microprocesadores de 16 bits desarrollados por la empresa, estableciendo un importante precedente en la industria tecnológica. El 8086 y el 8088 marcaron el inicio de la arquitectura x86, que se consolidaría como el estándar predominante en los procesadores de computadoras personales. Esta arquitectura evolucionaría considerablemente a lo largo de las décadas, influenciando generaciones posteriores de microprocesadores.



El Intel 8086 se destacó por tener un bus de datos de 16 bits, permitiendo un rendimiento superior en comparación con el 8088, cuyo bus de datos era de 8 bits. Esta diferencia hacía al 8088 más económico y compatible con el hardware existente de la época, lo que lo convirtió en la elección para la primera computadora personal de IBM lanzada en 1981. Pese a estas diferencias, ambos procesadores compartían un conjunto de instrucciones común y eran capaces de direccionar hasta 1 MB de memoria gracias a su innovador esquema de segmentación, que combinaba registros de segmento con desplazamientos de 16 bits para generar direcciones físicas de 20 bits. A su vez, el 8086 introdujo importantes mejoras técnicas respecto a sus predecesores, como el 8080 y el 8085. Incorporó registros de 16 bits más versátiles, una cola de prefetch de hasta seis bytes para optimizar la ejecución de instrucciones y una estructura que facilitaba la programación en lenguajes de alto nivel como Pascal y PL/M. Además, el 8086 podía operar en dos modos: modo mínimo, ideal para sistemas de un solo procesador, y modo máximo, diseñado para sistemas multiprocesador que empleaban coprocesadores como el 8087 para operaciones matemáticas avanzadas o el 8089 para control de entrada/salida.



La evolución de la arquitectura x86 continuó durante las siguientes décadas, destacando la llegada del Intel 80386 en 1985, que introdujo registros de 32 bits y un espacio de direcciones lineal más amplio. En 2017, Intel conmemoró el legado del 8086 con el lanzamiento del [Intel Core i7-8086K](#), una edición especial basada en la arquitectura Coffee Lake, que representó la 8ª generación de procesadores x86. Este modelo incluyó mejoras significativas en rendimiento, mayor número de núcleos (hasta seis en la gama i5) y optimización del consumo energético, reafirmando la relevancia histórica del 8086 como pionero de una familia de microprocesadores que sigue vigente en la actualidad.

[!NOTE]

El Intel 8086/8088, con su arquitectura x86, impulsó el ecosistema de [MS-DOS](#) (y luego Windows) en el mundo de las IBM PC y sus compatibles. Esta arquitectura ganó terreno principalmente en entornos empresariales y de productividad debido a la adopción masiva del estándar IBM PC. Hoy en día, los procesadores Intel modernos, como los de las familias Core i3, i5, i7, i9 y la nueva serie Core Ultra, siguen utilizando la arquitectura x86-64, que es una extensión de x86 que soporta instrucciones de 64 bits, introducida por AMD con el AMD64 y adoptada posteriormente por Intel como Intel 64.

CUADRO COMPARATIVO: 8086 vs 8088

Característica	Intel 8086	Intel 8088
Cola de instrucciones	6 bytes	4 bytes
Bus de direcciones	20 bits, permite direccionar hasta 1 MB de memoria	20 bits, permite direccionar hasta 1 MB de memoria
Bus de datos externo	16 bits, lo que permite transferir 16 bits de datos por ciclo de reloj	8 bits, lo que limita la transferencia de datos a 8 bits por ciclo de reloj
Costo y soporte	Más caro, requiere circuitos lógicos adicionales para manejar su bus de 16 bits	Más económico, compatible con hardware de 8 bits, más común en la época
Rendimiento	Más rápido debido a su bus de datos de 16 bits y mayor cola de instrucciones	Más lento debido a su bus de 8 bits y cola de instrucciones más corta
Tecnología de fabricación	Fabricado con tecnología de 3 micras (3 μm), con 29.000 transistores	Fabricado con tecnología de 3 micras (3 μm), con 29.000 transistores
Memoria direccionable	Hasta 1 MB, gracias a su arquitectura segmentada de 20 bits para direcciones	Hasta 1 MB, con el mismo esquema de direccionamiento segmentado
Impacto en el mercado	Menor adopción inicial debido a sus mayores costos y la necesidad de hardware especializado de 16 bits	Mayor adopción gracias a su compatibilidad con hardware de 8 bits, siendo elegido para la IBM PC en 1981
Relevancia histórica	Considerado más potente técnicamente, pero menos popular en sus primeros años	Se convirtió en el primer procesador usado en la IBM PC , la primera computadora personal de amplia adopción, lo que consolidó el éxito de Intel

- Tanto el iAPX 86 como el 88 utilizan la arquitectura x86, de tipo CISC (Complex Instruction Set Computing), que permite un conjunto amplio de instrucciones que realizan múltiples operaciones en un solo ciclo. Esto contrasta con las arquitecturas RISC (Reduced Instruction Set Computing), que tienen instrucciones más simples y requieren más ciclos para tareas complejas. Aunque CISC facilita la ejecución de programas complejos, su mayor complejidad de hardware y ciclos adicionales pueden ser una desventaja.
- El set de instrucciones es el mismo para ambos procesadores.
- Las frecuencias de reloj eran de 5, 8 y 10 MHz.

1. Microprocesador: es un tipo específico de procesador que está integrado en un solo chip de silicio. Todos los microprocesadores son procesadores, pero no todos los procesadores son microprocesadores. ↩

UNIDAD DE EJECUCIÓN (EU)

Esta unidad se encarga de decodificar las instrucciones de la cola de instrucciones, a través del [sistema de control](#), y realiza el tratamiento de calculo relacionado con la la unidad arimético-lógica (ALU)^[1], vinculando la instrucción con el conjunto de registros de propósito general^[2]. La EU se encarga de ejecutar las instrucciones que involucren cálculos, manipulación de datos y la toma de decisiones condicionales, dependiendo de los resultados obtenidos de la ALU. La comunicación entre la EU y la ALU es clave, ya que la ALU es responsable de realizar las operaciones que son el núcleo de las instrucciones que el procesador ejecuta, como sumas, restas, multiplicaciones, divisiones, y operaciones lógicas.

REGISTROS DE PROPOSITO GENERAL (GPR)

Registro AX (Acumulador)

Es el registro principal para operaciones aritméticas (como sumas, restas, multiplicaciones y divisiones) y de entrada/salida (I/O), cumpliendo un papel similar al que se tiene en lenguaje de máquina. Muchas instrucciones aritméticas y lógicas están optimizadas para trabajar directamente con AX, lo que lo convierte en el registro más utilizado para cálculos. Además, en operaciones de multiplicación y división, AX (y su extensión DX, para resultados de 32 bits) juega un papel central. Está optimizado para cálculos y operaciones de entrada/salida, lo que genera código más limpio, eficiente y fácil de entender. Usar otros registros como DX puede requerir instrucciones adicionales, lo que aumenta el tamaño del código y reduce su eficiencia.

Registro BX (Base)

Funciona como puntero de memoria y es el único que puede usarse para direccionamiento indexado. Esto significa que puede combinarse con registros de segmento (como DS o ES) para acceder a direcciones de memoria mediante un offset, lo que es clave en el manejo de punteros y estructuras de datos. Por ejemplo, en operaciones como `MOV AX, [BX + SI]`, BX actúa como base para calcular direcciones.

Registro CX (Contador)

Si bien se puede usar para cálculos, su diseño está optimizado para controlar repeticiones, ya que decrementa automáticamente su valor en cada iteración y, al llegar a 0, puede activar una señal que

permite salir del bucle. Esto lo hace ideal para estructuras como LOOP, donde simplifica la implementación de ciclos repetitivos en programas escritos en ensamblador.

Registro DX (De Datos)

Desempeña varias funciones dependiendo del contexto. En operaciones de 32 bits, como multiplicaciones y divisiones de 16 bits, DX se utiliza junto con AX para almacenar el resultado de 32 bits, donde AX contiene los 16 bits menos significativos y DX los 16 bits más significativos. Además, DX juega un papel clave en las operaciones de entrada/salida (I/O), ya que se usa para especificar el puerto al que se desea acceder, mientras que AX contiene los datos que se quieren enviar o recibir. Así, DX es esencial tanto en la manipulación de grandes números como en la comunicación con puertos de I/O.

REGISTROS DE DIRECCIONES DE DATOS (PUNTEROS)

Registro SP (Stack Pointer)

Es un registro especializado que actúa como puntero de pila y, aunque físicamente es de propósito general (pues se puede manipular como cualquier otro registro), su función principal está directamente relacionada con la gestión de la pila. El SP almacena la dirección de memoria que apunta a la cima (top) de la pila en el segmento de pila (SS). Cada vez que se realiza una operación PUSH, el valor en SP disminuye en 2 (debido a que el 8086 maneja datos en palabras de 16 bits) y el dato se almacena en esa nueva dirección. Por otro lado, al ejecutar una instrucción POP, se extrae el valor de la dirección apuntada por SP y luego este se incrementa en 2. Dado que la pila en el 8086 crece hacia abajo en memoria (es decir, hacia direcciones menores), estas modificaciones en SP permiten mantener el control correcto de los datos almacenados temporalmente y de las direcciones de retorno durante las llamadas a subrutinas.

Registro BP (Base Pointer)

Se emplea principalmente para el direccionamiento indirecto dentro de la pila, especialmente en el manejo de parámetros dentro de subrutinas. A diferencia del registro SP, que siempre apunta a la cima de la pila, el BP se utiliza para acceder a datos almacenados en posiciones relativas dentro de la pila, sin que su valor cambie automáticamente durante operaciones como PUSH o POP. Esto es útil porque permite navegar fácilmente por los elementos de la pila, como parámetros de funciones o valores locales. En este contexto, el BP se usa normalmente junto con el segmento de pila (SS) para

formar direcciones efectivas, facilitando el acceso ordenado a datos temporales o argumentos en subrutinas.

Registro SI (Source Index)

Sirve como puntero fuente para las operaciones con cadena^[3]. También sirve para realizar direccionamiento indirecto. Lleva la dirección donde comienza los datos que se quieren leer dentro de un segmento de datos. Almacena la dirección inicial de los datos que se desean leer dentro del segmento de datos (DS), y durante estas operaciones, su valor puede incrementarse o decrementarse automáticamente según la dirección del flujo de datos (controlado por el bit DF del registro de banderas). Además, el SI también se puede usar para realizar direccionamiento indirecto, lo que permite acceder a datos en posiciones variables dentro de un segmento.

Registro DI (Destination Index)

Se emplea como puntero de destino en las mismas operaciones con cadenas, apuntando a la dirección dentro del segmento de extra de datos (ES) donde se almacenarán los datos copiados o procesados. La combinación de SI y DI es esencial para mover eficientemente bloques de datos en memoria.

[!NOTE]

El BP está diseñado para trabajar principalmente con el Segmento de Pila (SS), mientras que el BX suele asociarse con el Segmento de Datos (DS). Por ejemplo:

- `MOV AX, [BP + SI]` : usa BP como base y SI como desplazamiento, pero apunta automáticamente al segmento de pila (SS) en lugar del segmento de datos (DS).
- `MOV AX, [BX + SI]` : usa BX como base y SI como desplazamiento, pero apunta automáticamente al segmento de datos (DS).

REGISTROS DE ESTADO (INDICADORES/FLAGS)

Contiene información de estado y de control de las operaciones del procesador. Aunque es de 16 bits, solo 9 de estos bits se utilizan para diferentes banderas (porque solo necesitaba esas banderas para su conjunto de instrucciones), que permiten tomar decisiones sobre el comportamiento de las instrucciones, especialmente en aquellas que implican bifurcaciones condicionales. El registro de indicadores se modifica automáticamente por la CPU después de realizar operaciones matemáticas y otras instrucciones.

Normalmente, no se puede acceder directamente al registro de flags de la misma manera que a otros registros de propósito general como AX, BX, etc. Las banderas son modificadas por las instrucciones

que realiza la CPU, y las condiciones de estas banderas determinan el comportamiento del programa. Sin embargo, es posible cambiar ciertos valores de las banderas del sistema (por ejemplo, las banderas de interrupción o dirección) usando instrucciones específicas, como la instrucción CLI (Clear Interrupt Flag) para deshabilitar las interrupciones, o STI (Set Interrupt Flag) para habilitarlas.

- **Banderas de Estado:**

- **OF (Overflow Flag):** se activa cuando hay un desbordamiento aritmético, es decir, cuando el resultado de una operación excede el rango que se puede representar con el número de bits disponibles. Esto ocurre principalmente en operaciones de suma o resta con números con signo.
- **SF (Sign Flag):** refleja el signo del resultado de una operación. Se activa si el resultado es negativo (es decir, si el bit más significativo del resultado es 1).
- **ZF (Zero Flag):** se activa si el resultado de una operación aritmética o lógica es cero. Es útil en operaciones de comparación o para verificar si el resultado de una operación fue cero.
- **AF (Auxiliar Flag):** Bandera auxiliar. Se utiliza principalmente en operaciones aritméticas con números BCD (Decimal Codificado en Binario). Indica si es necesario realizar un ajuste en los cálculos después de operar con valores en formato BCD, que es común en las aplicaciones financieras o de contabilidad.
- **PF (Parity flag):** indica si el número de bits con valor 1 en el resultado de una operación es par o impar. Se activa si hay un número par de bits en el resultado, y se utiliza generalmente para verificar la integridad de los datos transferidos entre componentes de la computadora.
- **CF (Carry Flag):** indica si ha ocurrido un acarreo o un préstamo en las operaciones aritméticas. Se activa cuando hay un acarreo de la operación aritmética, como en una suma que excede el valor máximo del tipo de dato.

- **Banderas de Control:**

- **DF (Direction Flag):** controla la dirección en la que se realizan las operaciones con cadenas de caracteres. Esta bandera determina si los registros índices SI (Source Index) y DI (Destination Index) se incrementan o se decrementan automáticamente durante las operaciones. Si DF está activada, los registros se decrementan (trabajando de atrás hacia adelante); si está desactivada, los registros se incrementan (trabajando de adelante hacia atrás).
- **IF (Interrupt Enable Flag):** indica si las interrupciones externas están habilitadas o no. Si está activada, el procesador acepta interrupciones externas, como las provenientes de dispositivos de entrada/salida o temporizadores. Si está desactivada, el procesador ignorará las interrupciones externas, lo cual puede ser útil en situaciones donde no se desea que se interrumpa el flujo de ejecución.
- **TF (Trap Flag):** permite ejecutar el procesador en modo paso a paso, de forma que, cuando está activada, el procesador ejecuta una instrucción y luego genera una interrupción de

software, lo que permite que el programa se detenga después de cada instrucción y sea inspeccionado. Es muy útil en herramientas de depuración como el programa DEBUG, ya que permite al programador examinar el estado del procesador después de cada operación y realizar un seguimiento detallado de la ejecución.

La distribución de las banderas en el registro de indicadores del 8086 no fue al azar, sino tiene una fuerte relación con los procesadores anteriores de Intel, como el 8080 y 8085. De hecho, muchas de las decisiones de diseño del 8086 se basaron en mantener compatibilidad hacia atrás con estos modelos. Esto se refleja en la ubicación de las flags dentro del registro, lo cual se puede observar al analizar los procesadores anteriores.

El objetivo no solo era garantizar la compatibilidad de software (permitiendo que los programas escritos para el 8080 y 8085 siguieran funcionando), sino también optimizar el uso de los recursos del procesador. Al hacerlo, las señales de control de las operaciones aritméticas y lógicas podían gestionarse de manera más eficiente, sin la necesidad de rediseñar por completo el hardware.

Bit	15-12	11	10	9	8	7	6	5	4	3	2	1	0
Flag		OF	DF	IF	TF	SF	ZF		AF		PF		CF

REGISTRO TEMPORAL

El registro temporal es un componente interno del procesador, invisible al programador, que la ALU utiliza durante ciertas operaciones aritméticas, lógicas o de comparación. Su función principal es almacenar temporalmente uno de los operandos o un resultado parcial para permitir que la ALU realice el cálculo sin modificar directamente los registros involucrados.

[!NOTE]

¿Cuándo se utiliza el registro temporal?

✔ Sí se usa en instrucciones que involucran la ALU, como: ADD, SUB, CMP, AND, OR, XOR, NEG.

✘ No se usa en instrucciones que solo copian o intercambian datos, como: MOV, XCHG, PUSH, POP.

SISTEMA DE CONTROL

El sistema de control en la Unidad de Ejecución (UE) es responsable de coordinar las operaciones internas del procesador durante la ejecución de las instrucciones. Este sistema interpreta las

instrucciones, genera las señales necesarias para activar componentes como la ALU, los registros y los flags, y gestiona el flujo de datos y el control de las operaciones. En el ciclo clásico fetch-decodificación-ejecución, el sistema de control asegura que las instrucciones se tomen de la memoria, se decodifiquen correctamente y se ejecuten de acuerdo con la lógica especificada, controlando las operaciones de la ALU, el acceso a los registros y la actualización de los flags según los resultados de las operaciones.

BUS

Dentro de la UE esta presente un bus de datos^[4], también conocido como "bus interno" o "ALU bus", que consta de 16 bits. Permite que se realicen operaciones de lectura y escritura dentro del procesador, transfiriendo datos de un registro a otro, o desde y hacia la memoria. Además, las instrucciones que se ejecutan en la UE utilizan este bus para mover los datos que están siendo procesados o que necesitan ser almacenados en la memoria o los registros. Justamente, dado que el 8086 tiene un procesador de 16 bits, la UE opera con datos de 16 bits durante las operaciones aritméticas y lógicas. Resumidamente, conecta la ALU con los registros y otros componentes internos.

1. La ALU es un componente clave del procesador y se encarga de realizar operaciones matemáticas (como sumas, restas, multiplicaciones) y lógicas (comparaciones, AND, OR, NOT). Trabaja con datos binarios y opera bajo el control de la Unidad de Control (CU). En 8086, no incluía operaciones de punto flotante, que requerían un coprocesador externo como el 8087. ↩
2. Los GPR (general purpose registers), son versátiles ya que pueden almacenar tanto datos (valores numéricos, resultados de operaciones) como direcciones de memoria (para acceder a la RAM). Además, se utilizan en operaciones aritméticas, lógicas, de comparación y manipulación de memoria, siendo esenciales para la ejecución eficiente de instrucciones en un programa. ↩
3. Son instrucciones diseñadas para trabajar con bloques de datos de forma rápida y eficiente. En lugar de tener que escribir un bucle manual para mover, comparar o buscar datos byte por byte, estas instrucciones lo hacen automáticamente usando registros específicos. ↩
4. Los buses se utilizan para interconectar varios registros, unos son considerados registros fuentes de información, mientras que otros son denominados como destinatarios de la información. ↩

UNIDAD DE INTERFAZ DEL BUS (BIU)

Es la encargada de gestionar la comunicación entre el procesador y los dispositivos externos, como la memoria y los periféricos, generando direcciones físicas a partir de los registros de segmento y el desplazamiento, realizando operaciones de lectura y escritura en la memoria, manteniendo una cola de instrucciones para optimizar el flujo de trabajo de la Unidad de Ejecución (EU), y controlando los buses del sistema para asegurar la correcta transferencia de datos e instrucciones. Básicamente, se encarga de leer y escribir datos e instrucciones, manejar el bus de direcciones y datos, y calcular direcciones físicas en el esquema de memoria segmentada. Además, carga instrucciones en la cola de instrucciones para que la Unidad de Ejecución las procese, permitiendo un prefetching que mejora la eficiencia al superponer la lectura y ejecución de instrucciones. En resumen, la BIU facilita el flujo de datos e instrucciones, optimizando el rendimiento del procesador.

REGISTROS DE DIRECCIONES DE SEGMENTOS

Los registros de segmento tienen un propósito muy específico: apuntar a bloques de memoria accesibles. Aunque técnicamente puedes almacenar cualquier dato en estos registros, no es recomendable hacerlo, ya que están diseñados para contener segmentos de memoria y permitir un acceso eficiente a la memoria segmentada.

Registro CS (Code Segment)

Apunta al segmento de memoria donde se almacenan las instrucciones del programa. Junto con el contador de programa (IP), que indica el offset dentro del segmento, el CS forma la dirección física de la siguiente instrucción a ejecutar. Esto permite al procesador acceder al código del programa de manera eficiente en el esquema de memoria segmentada del 8086. En resumen, el CS es esencial para direccionar y ejecutar correctamente las instrucciones de un programa.

Registro DS (Data Segment)

punta al segmento de memoria donde se almacenan los datos del programa, como las variables y otros valores que se utilizan durante la ejecución. Es decir, el DS generalmente señala el segmento en el que se encuentran las variables definidas en el programa. En un sistema de memoria segmentada como el 8086, el registro DS se utiliza para direccionar la ubicación de los datos que el procesador necesita para las operaciones. Esto incluye variables globales, arreglos y otros elementos de datos que no son instrucciones.

Registro ES (Extra Segment)

Su propósito es apuntar a un segmento de memoria adicional que se puede utilizar para almacenar información, como datos adicionales, que no se encuentran en los otros segmentos. A diferencia de los registros de segmento más específicos como CS o SS, este registro es más general y el ofrece flexibilidad sobre cómo usarlo. Comúnmente se emplea para almacenar información adicional o acceder a áreas de memoria fuera de los segmentos principales.

Registro SS (Stack Segment)

Apunta al segmento de memoria donde se encuentra la pila^[1] del programa. En 8086, el registro SS se utiliza específicamente para direccionar el segmento de pila, y permite al procesador realizar operaciones como push y pop para agregar o quitar datos de la pila. En el contexto del iAPX-86, la pila se encuentra en el segmento de pila direccionado por el registro SS.

- **Push:** La operación push agrega un valor al tope de la pila. Cuando se realiza un push, el registro SS y el registro SP (Puntero de Pila) se actualizan, y el valor se coloca en la dirección de memoria que indica el SP. Después de realizar la operación, el valor de SP se decrementa (ya que en el 8086 la pila crece hacia abajo, es decir, hacia direcciones de memoria más bajas).
- **Pop:** La operación pop extrae un valor desde el tope de la pila. En esta operación, el valor en la dirección de memoria señalada por el SP se lee, y luego el SP se incrementa para apuntar a la siguiente dirección disponible en la pila. Esto efectivamente "elimina" el valor del tope de la pila.

[!NOTE]

- BX, SI y DI se usan con DS para formar direcciones efectivas.
- BP y SP se usan con SS para formar direcciones efectivas.
- Otros registros de propósito general como AX, CX, DX, SI, DI también pueden formar direcciones efectivas, pero las partes alta y baja de BX no pueden formar direcciones efectivas por sí solas. BX en su conjunto sí puede ser utilizado para acceder a la memoria, pero BH y BL (por separado) no lo pueden hacer.

COLA DE INSTRUCCIONES (INSTRUCTION QUEUE)

Su función principal es almacenar temporalmente las instrucciones que el procesador va a ejecutar, permitiendo que el ciclo de **fetch** (obtención de instrucciones) y el de ejecución se solapen. Esto significa que mientras una instrucción se está ejecutando, la siguiente puede estar siendo leída del bus externo y almacenada en la cola, de manera que cuando la unidad de ejecución termine con la instrucción actual, ya tenga la siguiente lista para ejecutar. Este proceso de solapamiento aumenta la

eficiencia del procesador y reduce el tiempo de inactividad. Este paralelismo es el principio del [pipelining](#), que mejora el rendimiento del procesador al solapar etapas del ciclo de instrucción. Sin esta cola, el procesador debería esperar a que se complete la recuperación de cada instrucción antes de poder decodificarla o ejecutarla, lo que reduciría significativamente la velocidad.

Ciclo Fetch

El ciclo de instrucción, también conocido como ciclo fetch-decodificación-ejecución, es el proceso que realiza la CPU para obtener, interpretar y ejecutar instrucciones desde la memoria. Este ciclo consta de tres etapas principales: fetch (búsqueda), decode (decodificación) y execute (ejecución). El procesador 8086 implementa este ciclo optimizando el rendimiento mediante el uso de una cola de instrucciones (instruction queue) y un esquema de pipelining.

Tanto en el 8086 como en el 8088, el ciclo de fetch consta de varias etapas que involucran tanto al procesador como a la memoria:

1. **Generación de la dirección de memoria:** el procesador utiliza el contador de programa (IP) para determinar cuál es la dirección de la siguiente instrucción que debe ejecutar. La dirección física se calcula al combinar el segmento de :ódigo (CS) con el offset proporcionado por el IP, lo que permite obtener la ubicación exacta de la instrucción en la memoria segmentada.
2. **Solicitud a la memoria:** la BIU envía la dirección calculada al bus de direcciones para solicitar la instrucción desde la memoria.
3. **Lectura de la instrucción:** la memoria responde enviando los datos correspondientes (es decir, los bytes de la instrucción) al procesador a través del bus de datos.
4. **Recuperación de los bytes de la instrucción:** se almacenan los bytes recuperados en la cola de instrucciones, lo que permite que la EU decodifique y ejecute instrucciones mientras la BIU sigue recuperando nuevas instrucciones de la memoria, mejorando la eficiencia del procesamiento.
5. **Incremento del contador de programa:** IP se incrementa para apuntar a la siguiente instrucción en la memoria, asegurando que el procesador obtenga la instrucción correcta en el siguiente ciclo.

Ahora bien, el **pipelining** es una técnica que permite que varias etapas del ciclo de instrucción se ejecuten en paralelo. Mientras una instrucción se decodifica, otra puede estar siendo ejecutada y una tercera puede estar siendo recuperada desde la memoria. Este enfoque es posible gracias a la cola de instrucciones, que actúa como un búfer que desacopla la recuperación de instrucciones de su ejecución. Este mecanismo es una característica clave de los procesadores CISC, como el 8086 y el 8088, ya que estos procesadores suelen manejar instrucciones más complejas que requieren mayor tiempo de decodificación y ejecución.

Sin embargo, si se produce un evento que altere el flujo normal del programa, como una instrucción de salto (JMP^[2]), una llamada a subrutina (CALL^[3]) o una interrupción, la cola de instrucciones se vacía automáticamente. Esto se debe a que, tras un cambio en el flujo del programa, las instrucciones previamente almacenadas ya no son válidas, y el procesador debe cargar nuevas instrucciones desde la nueva dirección en memoria.

[!NOTE]

En 8086 este bus tiene 6 bytes, mientras que en 8088 es solo de 4 bytes.

CONTADOR DE PROGRAMA

El Contador de Programa (IP) es un registro crucial que contiene el offset dentro del segmento de código, indicando la posición de la siguiente instrucción a ejecutar. El IP por sí solo no contiene la dirección completa, sino solo el desplazamiento dentro de un segmento específico de memoria. Este registro se incrementa en 1 después de cada instrucción ejecutada, para señalar la próxima ubicación en el segmento de código.

Cuando se combina el CS (registro de segmento de código) con el IP, se obtiene la dirección física completa de la instrucción a ejecutar. El CS apunta al inicio del segmento de código, y el IP determina la posición exacta de la instrucción dentro de ese segmento. La dirección física se calcula multiplicando el valor de CS por 16 (equivalente a desplazamiento de 4 bits a la izquierda) y sumando el IP. Esto permite que el procesador acceda a la instrucción en la memoria.

BUSES

En cuanto a los buses:

- El bus de direcciones tiene una longitud de 20 bits, lo que permite acceder a una memoria de hasta 1 MB (2^{20} direcciones). La dirección física se obtiene combinando el valor del segmento (CS, DS, SS, ES) y el desplazamiento (offset). Esta presente solo en la BIU.
- El bus de datos varía según el procesador: el 8088 tiene un bus de datos de 8 bits, mientras que el 8086 cuenta con un bus de 16 bits. Esto significa que el 8086 puede transferir más datos por ciclo de reloj, lo que le da una ventaja en términos de velocidad de transferencia de datos en comparación con el 8088. Básicamente, es un bus de datos de la BIU que conecta con la cola de instrucciones.

1. La pila es una estructura LIFO (Last in, first out) de memoria utilizada para almacenar datos temporales, como las direcciones de retorno de las subrutinas, variables locales y otros datos necesarios durante la ejecución del programa. ↩
2. Esta instrucción hace que el procesador salte a una nueva dirección de memoria, lo que cambia la secuencia de ejecución del programa. Puede ser un salto incondicional, es decir, siempre salta a la dirección indicada. Esto es útil para bucles o saltos a otras partes del código. ↩
3. Esta instrucción se utiliza para llamar a una subrutina o función. Al ejecutarse, el procesador guarda la dirección de la instrucción siguiente (la que sigue al CALL) en la pila, para que pueda regresar a ella después de ejecutar la subrutina. Luego, salta a la dirección de la subrutina indicada. Después de ejecutar la subrutina, se usa RET (Return) para volver a la instrucción guardada en la pila. ↩

MODOS DE DIRECCIONAMIENTO

Son técnicas que utilizan las instrucciones de un procesador para determinar cómo se calcula la dirección de memoria de un operando. Es decir, son las estrategias que permiten a las instrucciones de la CPU acceder a los datos que necesitan para realizar operaciones. El modo de direccionamiento especifica la forma en que se obtiene la dirección de memoria efectiva de un operando, utilizando información como registros, constantes o la propia instrucción de la máquina. Los modos de direccionamiento pueden involucrar registros, desplazamientos, valores inmediatos, entre otros, y pueden variar dependiendo de la arquitectura del procesador.

En el microprocesador Intel 8086, se implementan varios modos de direccionamiento que permiten la flexibilidad para acceder a los datos de diferentes maneras. Aquí se explican algunos de los modos más relevantes que se utilizan en esta arquitectura:

DIRECCIONAMIENTO DE REGISTRO (REGISTER ADDRESSING)

Es el más sencillo de todos, y consiste en mover los contenidos de un registro a otro, es decir, los dos operandos son registros. Por ejemplo, si quiero mover el contenido del registro de datos al acumulador.

Ejemplo:

```
MOV AX, DX
```

DIRECCIONAMIENTO INMEDIATO (IMMEDIATE ADDRESSING)

En este modo, el operando está directamente especificado en la instrucción, es decir, el valor a usar es parte de la instrucción misma. Básicamente, el operando a almacenar es una constante.

Ejemplo:

```
MOV AX, 5
```

DIRECCIONAMIENTO NORMAL/DIRECTO (DIRECT ADDRESSING)

La dirección del operando está contenida directamente en la instrucción. Especifica la dirección absoluta en la memoria, de forma que el operando es una dirección de memoria.

Ejemplo:

```
MOV AX, [5h]; Mediante [] indica que se carga el contenido de la direccion dad  
MOV AX, variable; El orgien es una variable definida anteriormente que contiene el valor a almac
```

DIRECCIONAMIENTO INDIRECTO (INDIRECT ADDRESSING)

Se basa en el uso de registros para localizar un operando en memoria. En este modo, la dirección de memoria efectiva (donde se encuentra el operando) está almacenada en un registro, lo que introduce un proceso en dos etapas.

En el primer ciclo, la CPU lee el contenido del registro que actúa como puntero, el cual contiene la dirección de memoria efectiva. En el segundo ciclo, utiliza esa dirección para acceder a la memoria y obtener el operando. Es decir, el registro no contiene directamente el operando, sino que guarda la dirección donde se encuentra dicho operando.

Ejemplo:

```
MOV AX, [BX]
```

- **Primer ciclo:** el procesador accede al contenido del registro BX, que contiene una dirección de memoria, por ejemplo 2000h.
- **Segundo ciclo:** la CPU accede a la dirección 2000h en la memoria para obtener el valor almacenado allí y lo carga en el registro AX.

DIRECCIONAMIENTO POR REGISTRO BASE (BASE ADDRESSING)

Se utiliza cuando el operando está en memoria en una posición apuntada por un registro base, como BX o BP, al que se le añade un desplazamiento. Este modo permite acceder de forma eficiente a

estructuras de datos que se encuentran en memoria.

En este modo, la dirección efectiva se calcula así:

$$\text{Dirección efectiva} = \text{Valor del registro base} + \text{Desplazamiento}$$

- BX → Usado principalmente para acceder a datos en el segmento de datos (DS).
- BP → Usado principalmente para acceder a datos en el segmento de pila (SS).

Ejemplo:

```
MOV AX, [BX + 5] ; La dirección efectiva será el valor de BX más 5
MOV CX, [BP + 10h] ; La dirección efectiva será el valor de BP más 10h
```

[!NOTE]

Este modo es similar al direccionamiento indexado, pero se diferencia en que emplea registros específicos (BX o BP) que suelen estar asociados a segmentos de datos o pila, respectivamente.

DIRECCIONAMIENTO CON DESPLAZAMIENTO/INDEXADO (INDEXED ADDRESSING)

Es una técnica que calcula la dirección efectiva del operando sumando un valor constante (desplazamiento) a un registro que actúa como índice (como SI o DI). Este modo es especialmente útil para recorrer estructuras de datos organizadas en memoria, como arreglos, tablas o cadenas de texto.

En este modo, la dirección efectiva se calcula así:

$$\text{Dirección efectiva} = \text{Valor del registro índice} + \text{Desplazamiento}$$

Los registros típicos que se emplean como índice son:

- SI → Índice fuente, generalmente usado para recorrer datos de origen.
- DI → Índice destino, generalmente usado para recorrer datos de destino.

Ejemplo:

```
MOV AX, [SI + 4] ; Se suma el valor del registro SI más 4 para obtener la dirección efectiva
MOV BX, [DI + 10h] ; Se suma el valor del registro DI más 10h para obtener la dirección efectiva
```

DIRECCIONAMIENTO CON DESPLAZAMIENTO INDEXADO RESPECTO A UNA BASE

En este caso, la dirección del operando se calcula a partir de la suma de tres componentes:

1. **Registro base:** Usualmente el registro `BP` o `BX`, que apunta a una base de datos o dirección en memoria.
2. **Índice:** Un registro índice, como `DI` o `SI`, utilizado para acceder a una posición específica dentro de una estructura de datos.
3. **Desplazamiento:** Un valor constante opcional que puede sumarse al cálculo, lo que permite acceder a posiciones específicas dentro de la estructura de datos.

Ecuación de direccionamiento indexado con respecto a una base

La dirección efectiva se calcula de la siguiente forma:

$\text{Dirección efectiva} = \text{Valor del registro base} + \text{Valor del registro índice} + \text{Desplazamiento}$

Ejemplo:

```
MOV AX, [BX + SI + 5]
```

DIRECCIONAMIENTO RELATIVO (RELATIVE ADDRESSING)

Se utiliza principalmente en las instrucciones de salto, como `JMP`, `JE`, `JNE`, `CALL`, entre otras. En este modo, la dirección del operando no está dada directamente, sino que se calcula sumando un valor inmediato (un desplazamiento) a la dirección de la instrucción siguiente. Este desplazamiento es generalmente un valor que indica cuántas posiciones hacia adelante o hacia atrás se debe saltar desde la instrucción actual.

El valor del operando (dirección de salto) se calcula como la dirección de la instrucción siguiente más un desplazamiento relativo, y puede ser tanto positivo como negativo, lo que permite realizar saltos hacia adelante o hacia atrás en el código. Es muy eficiente en las instrucciones de salto condicional y no condicional, ya que no requiere de una dirección absoluta en la memoria.

Ejemplo:

```
JMP +10 ; Salta 10 bytes hacia adelante desde la instrucción siguiente
JMP -20 ; Salta 20 bytes hacia atrás desde la instrucción siguiente
JE +5    ; Salta 5 bytes hacia adelante si la condición es verdadera
```

DIRECCIONAMIENTO BASADO EN PILA (STACK ADDRESSING)

Este modo utiliza la pila para acceder a los datos. Las instrucciones relacionadas con la pila, como `PUSH` y `POP`, trabajan con este tipo de direccionamiento. En este modo, las operaciones se realizan sobre el tope de la pila, y no se especifican direcciones de memoria explícitas en las instrucciones. Como ya se mencionó antes, la pila en el 8086/8088 funciona bajo el esquema LIFO, es decir, el último valor que se introduce es el primero que se extrae. La dirección de la pila se controla mediante los registros `SS` (Segmento de pila) y `SP` (Puntero de pila), lo que permite acceder a la dirección exacta del tope de la pila.

El comportamiento de la pila en el 8086 es así:

- **PUSH:** Guarda un valor en la pila. Primero el puntero de pila (`SP`) se decrementa en 2 (ya que en el 8086 cada palabra son 2 bytes) y luego el valor se almacena en la dirección indicada por `SS:SP`.
- **POP:** Extrae el valor que está en la cima de la pila. Primero el valor en `SS:SP` se transfiere al destino (como un registro o dirección de memoria), y luego el puntero de pila (`SP`) se incrementa en 2.

Ejemplo:

```
MOV AX, 1234h ; AX = 1234h
PUSH AX       ; Guarda 1234h en la pila (SP se decrementa en 2)
MOV BX, 5678h ; BX = 5678h
PUSH BX       ; Guarda 5678h en la pila (SP se decrementa en 2)
POP CX        ; CX = 5678h (SP se incrementa en 2)
POP DX        ; DX = 1234h (SP se incrementa en 2)
```